OCOM5300M Artificial Intelligence Project Conceptual Neuro-Symbolic Conformal Pipeline

In [1]:

```python
# custom classes
from conceptual_neurosymbolic_conformal_pipeline import (
    ConceptIndex,
    ROCOv2_ConceptDataset,
    ConceptClassifierModel,
    UMLSCUISymbolicRuleGenerator,
    UMLSSymbolicValidator,
    ConformalCalibrator,
    ConceptualNeuroSymbolicConformalPipeline
)

# deep learning Functionality
import torch  # PyTorch library

# batching, shuffling and data loading capabilities
from torch.utils.data import DataLoader

# data handling and analysis
import pandas as pd

# utilities
import os  # file and directory operations

# file and directory paths
from pathlib import Path

# handling csv files
import csv

# frequencies
from collections import Counter

# pretrained Medical Model, i.e., OpenCLIP implementation used to load BiomedCLIP
import open_clip
```

In [2]:
```
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```
# detect device (CPU/GPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

In [4]:
```
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [5]:
```
# set random seed to 42
seed = 42
```

In [6]:
```
# set PyTorch random number generator seed for reproducible results
torch.manual_seed(seed)
```

Out[6]:
```
<torch._C.Generator at 0x1c164c8d9f0>
```

In [7]:
```
# seed all available CUDA devices if using GPU
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

Pipeline Configuration

In [8]:
```
# set pipeline mode to process training and validation images or test images, i.e., "train_val" for testing / validation and "test"
# for held-out test set
pipeline_mode = "test"
```

In [9]:
```
# set model name - BiomedCLIP model, available from Hugging Face through OpenCLIP
biomedclip_model_name = "hf-hub:microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224"
```

In [10]:
```
# BiomedCLIP image embedding dimension
biomedclip_feature_dimension = 512
```

In [11]:
```
# determine whether pretrained BiomedCLIP visual model can be fine-tuned (for CUI classification)
freeze_pretrained_model = False
```

In [12]:
```
# include UMLS CUI concepts that appear in at least one ROCOv2 training imagege
minimum_concept_frequency = 1
```

In [13]:
```
# apply 20% dropout during training to help reduce overfitting
dropout = 0.2
```

In [14]:
```
# set GPU and CPU batch sizes (dependent on run-time environment)
batch_size = 16 if device.type == "cuda" else 4
```

In [15]:
```
# set number of training epochs for the concept classifier
num_epochs = 3
```

In [16]:
```
# set the learning rate used to update concept classifier weights during training
learning_rate = 0.00001
```

In [17]:
```
# apply a small penalty to large weights to help reduce overfitting
weight_decay = 0.0001
```

In [18]:
```
# stop training if validation loss does not improve for 2 consecutive epochs
patience = 2
```

In [19]:
```
# set number of PyTorch background worker processes used to load dataset batches
num_workers = 1
```

In [20]:
```
# Limit gradient magnitude during training to help prevent unstable weight updates
clip_grad_norm = 1.0
```

In [21]:
```
# retain predicted CUI concepts with a probability of at least 50%
minimum_prediction_probability = 0.5
```

In [22]:
```
# pipeline retention threshold, i.e. for output files
pipeline_retention_threshold = 0.8
```

In [23]:
```
# reuse the saved classifier checkpoint unless retraining is explicitly requested
force_retrain_concept_classifier = False #True
```

In [24]:
```
# set the conformal error level to 5%, i.e. the an acceptance threshold for each CUI concept
# to reduce false detections of that specific concept
conformal_alpha = 0.05
```

In [25]:
```
# require at least 20 calibration images where ground-truths do not
# contain a specific CUI. This ensures that there are enough negative examples to estimate
# the probability threshold used to accept or reject predictions for that CUI
minimum_calibration_examples = 20
```

In [26]:
```
# require symbolic rules derived from UMLS relationships
require_symbolic_rules = True
```

In [27]:
```
# set file and folder paths
training_images_folder = "ROCOv2/train_images/train"
validation_images_folder = "ROCOv2/valid_images/validation"
calibration_images_folder = "ROCOv2/calibration_images/calibration"
test_images_folder = "ROCOv2/test_images/test"
training_captions_filename = "ROCOv2/train_captions.csv"
validation_captions_filename = "ROCOv2/valid_captions_split.csv"
calibration_captions_filename = "ROCOv2/calibration_captions_split.csv"
test_captions_filename = "ROCOv2/test_captions.csv"
training_concepts_filename = "ROCOv2/train_concepts.csv"
validation_concepts_filename = "ROCOv2/valid_concepts.csv"
cui_mapping_filename = "ROCOv2/cui_mapping.csv"
concept_index_filename = "ROCOv2/biomedclip_concept_index.pt"
concept_classifier_checkpoint = "ROCOv2/biomedclip_concept_classifier_best.pth"
calibration_rules_filename = "ROCOv2/conformal_calibration_rules.csv"
pipeline_output_folder = "ROCOv2"
symbolic_rules_filename = "ROCOv2/rocov2_symbolic_rules_train_only.csv"
```

Load ROCOv2 Metadata

In [28]:
```
def load_required_csv(filename):
    """
    Load required CSV file

    Args:
        filename (str): CSV file path
```

```
    Returns:
        pd.DataFrame: loaded CSV data
    """

    try:

        # check file exists
        if not Path(filename).exists():
            raise FileNotFoundError(f"File not found: {filename}")

        # load CSV file
        return pd.read_csv(filename)

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [29]:
```
# load CUI and concept name mapping files
cui_mapping = load_required_csv(cui_mapping_filename)
```

In [30]:
```
# load metadata required for selected pipeline (mode)
if pipeline_mode == "train_val":

    # load training and revised validation/calibration metadata
    training_captions = load_required_csv(training_captions_filename)
    validation_captions = load_required_csv(validation_captions_filename)
    calibration_captions = load_required_csv(calibration_captions_filename)
    training_concepts = load_required_csv(training_concepts_filename)
    original_validation_concepts = load_required_csv(validation_concepts_filename)

elif pipeline_mode == "test":

    # load test metadata used only to identify test images
    test_captions = load_required_csv(test_captions_filename)
```

Prepare Pipeline Datasets

In [31]:
```
# prepare image IDs and concept data for training, validation and calibration
if pipeline_mode == "train_val":

    # image IDs
```

```
    training_ids = training_captions["ID"].astype(str).drop_duplicates().tolist()
    validation_ids = validation_captions["ID"].astype(str).drop_duplicates().tolist()
    calibration_ids = calibration_captions["ID"].astype(str).drop_duplicates().tolist()

    # convert validation image IDs to strings to match against validation and calibration image IDs
    original_validation_concepts["ID"] = original_validation_concepts["ID"].astype(str)

    # retain concept labels for the validation set
    validation_concepts =
original_validation_concepts[original_validation_concepts["ID"].isin(validation_ids)].copy().reset_index(drop=True)

    # retain concept labels for the calibration set
    calibration_concepts =
original_validation_concepts[original_validation_concepts["ID"].isin(calibration_ids)].copy().reset_index(drop=True)

    # display split sizes
    print(f"Training images: {len(training_ids)}")
    print(f"Validation images: {len(validation_ids)}")
    print(f"Calibration images: {len(calibration_ids)}")
```

In [32]:
```
# prepare test image IDs
if pipeline_mode == "test":

    # test image IDs
    test_ids = test_captions["ID"].astype(str).drop_duplicates().tolist()

    # display test split size
    print(f"Test images: {len(test_ids)}")
```

```
Test images: 9927
```

Stage 1 - Conceptual Pipeline

Concept Index

In [33]:
```
# create / load each UMLS CUI concept index mapping, i.e., to a classifier class number
if pipeline_mode == "train_val":

    # instantiate concept index
    concept_index = ConceptIndex()
```

```
    # build index from training CUIs
    concept_index.build_index(training_concepts, minimum_concept_frequency=minimum_concept_frequency)

    # add canonical UMLS concept names
    concept_index.add_concept_names(cui_mapping)

    # save concept index for test mode
    concept_index.save(concept_index_filename)

    # display number of CUI classes
    print(f"Concept classes: {len(concept_index)}")
```

In [34]:
```
if pipeline_mode == "test":

    # load concept index created during training and validation mode
    concept_index = ConceptIndex.load(concept_index_filename)

    # display number of CUI classes
    print(f"Concept classes: {len(concept_index)}")
```

```
Concept classes: 1947
```

BiomedCLIP ViT-B/16

In [35]:
```
# load pretrained BiomedCLIP model and image transformations
biomedclip_model, biomedclip_training_transform, biomedclip_validation_transform =
open_clip.create_model_and_transforms(biomedclip_model_name)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
nd faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
TOKEN to enable higher rate limits and faster downloads.
```

In [36]:
```
# only retain BiomedCLIP ViT-B/16 visual model
biomedclip_visual_model = biomedclip_model.visual
```

In [37]:
```
# remove unused text model reference
del biomedclip_model
```

In [38]:
```
# move pretrained visual model to selected device
```

```
biomedclip_visual_model = biomedclip_visual_model.to(device)
```

Concept Classification Datasets

In [39]:
```
# create data loaders for selected pipeline mode
if pipeline_mode == "train_val":

    # training dataset uses BiomedCLIP training transformation
    training_dataset = ROCOv2_ConceptDataset(training_images_folder, ".jpg", biomedclip_training_transform,
training_ids, concept_index,
                                             training_concepts)

    # deterministic training prediction dataset uses BiomedCLIP validation transformation
    training_prediction_dataset = ROCOv2_ConceptDataset(training_images_folder, ".jpg",
biomedclip_validation_transform, training_ids,
                                                        concept_index, training_concepts)

    # validation and calibration datasets use BiomedCLIP validation transformation
    validation_dataset = ROCOv2_ConceptDataset(validation_images_folder, ".jpg", biomedclip_validation_transform,
validation_ids,
                                               concept_index, validation_concepts)
    calibration_dataset = ROCOv2_ConceptDataset(calibration_images_folder, ".jpg", biomedclip_validation_transform,
calibration_ids,
                                                concept_index, calibration_concepts)

    # create data loaders
    training_loader = DataLoader(training_dataset, batch_size=batch_size, shuffle=True, num_workers=num_workers,
                                 pin_memory=(device.type == "cuda"))
    training_prediction_loader = DataLoader(training_prediction_dataset, batch_size=batch_size, shuffle=False,
                                            num_workers=num_workers, pin_memory=(device.type == "cuda"))
    validation_loader = DataLoader(validation_dataset, batch_size=batch_size, shuffle=False,
                                   num_workers=num_workers, pin_memory=(device.type == "cuda"))
    calibration_loader = DataLoader(calibration_dataset, batch_size=batch_size, shuffle=False,
num_workers=num_workers,
                                    pin_memory=(device.type == "cuda"))
```

In [40]:
```
if pipeline_mode == "test":

    # test dataset does not use test ground-truth concepts
    test_dataset = ROCOv2_ConceptDataset(test_images_folder, ".jpg", biomedclip_validation_transform, test_ids,
concept_index, concepts=None)
```

```
    # create test data loader
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=num_workers,
pin_memory=(device.type == "cuda"))
```

Concept Classifier

In [41]:
```
# instantiate BiomedCLIP multi-label CUI concept classifier
concept_classifier = ConceptClassifierModel(pretrained_model=biomedclip_visual_model,
feature_dimension=biomedclip_feature_dimension,
                                            num_concepts=len(concept_index), dropout=dropout,
freeze_pretrained_model=freeze_pretrained_model,
                                            feature_method="forward", pooling="mean").to(device)
```

In [42]:
```
# train or reload concept classifier when processing training and validation images
if pipeline_mode == "train_val":

    # reload saved best checkpoint
    if Path(concept_classifier_checkpoint).exists() and not force_retrain_concept_classifier:
        concept_classifier.load_checkpoint(concept_classifier_checkpoint, device)
        concept_classifier_history = concept_classifier.history
        print(f"Loaded concept-classifier checkpoint: {concept_classifier_checkpoint}")

    else:
        # calculate class weights to account for CUI class imbalance
        positive_weights = concept_index.calculate_positive_weights(training_concepts, training_ids)

        # train concept classifier and retain best validation checkpoint
        concept_classifier_history = concept_classifier.train_model(training_loader, validation_loader,
num_epochs=num_epochs,
                                                                     learning_rate=learning_rate,
weight_decay=weight_decay,
                                                                     patience=patience,
clip_grad_norm=clip_grad_norm,
                                                                     checkpoint_path=concept_classifier_checkpoint,
device=device,
                                                                     positive_weights=positive_weights)

        # confirm saved checkpoint
        print(f"Saved concept-classifier checkpoint: {concept_classifier_checkpoint}")
```

```
    # display training history
    display(concept_classifier_history)
```

In [43]:
```
# load trained concept classifier when processing test images
if pipeline_mode == "test":

    # load best concept classifier checkpoint
    concept_classifier.load_checkpoint(concept_classifier_checkpoint, device)
```

Generate CUI Predictions

In [44]:
```
# generate CUI probability predictions
if pipeline_mode == "train_val":

    # generate training predictions
    training_predictions = concept_classifier.generate_predictions(training_prediction_loader, concept_index,
device,

minimum_probability=minimum_prediction_probability)

    # generate validation predictions
    validation_predictions = concept_classifier.generate_predictions(validation_loader, concept_index, device,

minimum_probability=minimum_prediction_probability)

    # generate calibration predictions for diagnostics
    calibration_predictions = concept_classifier.generate_predictions(calibration_loader, concept_index, device,

minimum_probability=minimum_prediction_probability)

    # display prediction counts
    print(f"Training prediction rows: {len(training_predictions)}")
    print(f"Validation prediction rows: {len(validation_predictions)}")
    print(f"Calibration prediction rows: {len(calibration_predictions)}")
```

In [45]:
```
if pipeline_mode == "train_val":

    # diagnose prediction counts at different probability thresholds
    diagnostic_thresholds = [0.50, 0.60, 0.70, 0.80, 0.90, 0.95]
```

```
    diagnostic_results = []

    for split_name, predictions, total_images in [
        ("Training", training_predictions, len(training_ids)),
        ("Validation", validation_predictions, len(validation_ids)),
        ("Calibration", calibration_predictions, len(calibration_ids))
    ]:

        for threshold in diagnostic_thresholds:

            # retain predictions meeting diagnostic threshold
            filtered_predictions = predictions[
                predictions["Probability CUI In The Image"] >= threshold
            ]

            # calculate predictions per image
            predictions_per_image = filtered_predictions.groupby("Image ID").size()

            # store diagnostic results
            diagnostic_results.append({
                "Split": split_name,
                "Threshold": threshold,
                "Rows": len(filtered_predictions),
                "Average CUIs Per Image": len(filtered_predictions) / total_images,
                "Median CUIs Per Image": predictions_per_image.median() if not predictions_per_image.empty else 0,
                "Maximum CUIs Per Image": predictions_per_image.max() if not predictions_per_image.empty else 0
            })

    # create diagnostic table
    prediction_diagnostics = pd.DataFrame(diagnostic_results)

    # display results
    display(prediction_diagnostics)
```

In [46]:

```
if pipeline_mode == "test":

    # generate test predictions
    test_predictions = concept_classifier.generate_predictions(test_loader, concept_index, device,
minimum_probability=minimum_prediction_probability)
```

```
    # display prediction count
    print(f"Test prediction rows: {len(test_predictions)}")
```

```
Concept predictions:   0%|          | 0/2482 [00:16<?, ?it/s]
Test prediction rows: 1823618
```

Stage 2 - Neuro-Symbolic Pipeline

UMLS Symbolic Knowledge

In [47]:
```
# instantiate UMLS symbolic validator
symbolic_validator = UMLSSymbolicValidator(concept_index)
```

In [48]:
```
# generate UMLS symbolic rules using training concepts
if pipeline_mode == "train_val":

    # initialise UMLS symbolic rule generator
    umls_rule_generator = UMLSCUISymbolicRuleGenerator(concept_index=concept_index)

    # generate training-only rules using direct and two-step UMLS relationships
    symbolic_rules = umls_rule_generator.create_automatic_training_rules(
        training_concepts=training_concepts,
        output_folder="ROCOv2",
        minimum_support=10,
        minimum_confidence=0.98,
        minimum_lift=1.5,
        require_umls_relationship=True,
        umls_mrrel_filename="ROCOv2/MRREL.RRF",
        maximum_umls_hops=2
    )
```

In [49]:
```
# load symbolic rules
symbolic_validator.load_rules(symbolic_rules_filename)
```

Out[49]:
```
<conceptual_neurosymbolic_conformal_pipeline.UMLSSymbolicValidator at 0x1c174f9c6e0>
```

In [50]:
```
if pipeline_mode == "train_val":

    # validate training predictions
    training_symbolic_predictions = symbolic_validator.validate_predictions(training_predictions)
```

```
    # validate validation predictions
    validation_symbolic_predictions = symbolic_validator.validate_predictions(validation_predictions)

    # validate calibration predictions
    calibration_symbolic_predictions = symbolic_validator.validate_predictions(calibration_predictions)
```

In [51]:
```
if pipeline_mode == "test":

    # validate test predictions
    test_symbolic_predictions = symbolic_validator.validate_predictions(test_predictions)
```

Stage 3 - Conformal Pipeline

Conformal Calibration

In [52]:
```
# create or load conformal calibration rules
if pipeline_mode == "train_val":

    # instantiate conformal calibrator
    conformal_calibrator = ConformalCalibrator(alpha=conformal_alpha,
minimum_calibration_examples=minimum_calibration_examples)

    # create CUI-specific conformal thresholds from held-out calibration images
    calibration_rules = conformal_calibrator.fit(concept_classifier, calibration_loader, concept_index, device)

    # save calibration rules for test mode
    conformal_calibrator.save(calibration_rules_filename)

    # display calibration rules
    display(calibration_rules.head(10))
```

In [53]:
```
if pipeline_mode == "test":

    # load calibration rules created using held-out calibration images
    conformal_calibrator = ConformalCalibrator.load(calibration_rules_filename)
```

Apply Conformal Rules

In [54]:
```python
# apply conformal rules
if pipeline_mode == "train_val":

    # apply rules to symbolically validated training predictions
    training_conformal_predictions = conformal_calibrator.apply(training_symbolic_predictions)

    # apply rules to symbolically validated validation predictions
    validation_conformal_predictions = conformal_calibrator.apply(validation_symbolic_predictions)
```

In [55]:
```python
if pipeline_mode == "test":

    # apply saved rules to symbolically validated test predictions
    test_conformal_predictions = conformal_calibrator.apply(test_symbolic_predictions)
```

Combined Concept Reliability

In [56]:
```python
def add_reliability(predictions):
    """
    Assign final concept reliability

    Args:
        predictions (DataFrame): predictions containing symbolic and conformal validity

    Returns:
        pd.DataFrame: predictions with Reliable flag
    """

    try:

        # copy predictions
        reliable_predictions = predictions.copy()

        # mark a CUI prediction as reliable only if it passes both the symbolic rules
        # and the conformal probability threshold
        reliable_predictions["Reliable"] = reliable_predictions["Symbolic (Valid)"] &
reliable_predictions["Conformal (Valid)"]

        return reliable_predictions
```

```
    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [57]:
```
# assign final reliability rating
if pipeline_mode == "train_val":

    # training reliability
    training_reliable_predictions = add_reliability(training_conformal_predictions)

    # validation reliability
    validation_reliable_predictions = add_reliability(validation_conformal_predictions)
```

In [58]:
```
if pipeline_mode == "test":

    # test reliability
    test_reliable_predictions = add_reliability(test_conformal_predictions)
```

Create Pipeline DataFrames

In [59]:
```
# instantiate pipeline controller
pipeline_controller = ConceptualNeuroSymbolicConformalPipeline(mode=pipeline_mode)
```

In [60]:
```
# create final pipeline DataFrames
if pipeline_mode == "train_val":

    # create training pipeline
    training_pipeline = pipeline_controller.create_pipeline(training_predictions, symbolic_validator,
conformal_calibrator)

    # create validation pipeline
    validation_pipeline = pipeline_controller.create_pipeline(validation_predictions, symbolic_validator,
conformal_calibrator)
```

In [61]:
```
if pipeline_mode == "test":

    # create test pipeline
    test_pipeline = pipeline_controller.create_pipeline(test_predictions, symbolic_validator, conformal_calibrator)
```

Export Pipeline Files

In [62]:

```python
# export training and validation pipeline CSV files
if pipeline_mode == "train_val":

    # export complete training pipeline including rejected predictions
    training_pipeline_filename = pipeline_controller.save_pipeline(
        training_pipeline,
        pipeline_output_folder,
        "training"
    )

    # export complete validation pipeline including rejected predictions
    validation_pipeline_filename = pipeline_controller.save_pipeline(
        validation_pipeline,
        pipeline_output_folder,
        "validation"
    )

    # select training predictions meeting the retention threshold
    training_retained_predictions = training_predictions[
        training_predictions["Probability CUI In The Image"] >= pipeline_retention_threshold
    ].copy()

    # reapply symbolic and conformal validation to retained training predictions
    training_retained_pipeline = pipeline_controller.create_pipeline(
        training_retained_predictions,
        symbolic_validator,
        conformal_calibrator
    )

    # retain reliable training predictions
    training_accepted_pipeline = training_retained_pipeline[
        training_retained_pipeline["Reliable"]
    ].reset_index(drop=True)

    # save accepted training predictions separately
    training_accepted_filename = Path(pipeline_output_folder) / "pipeline_training_accepted.csv"
    training_accepted_pipeline.to_csv(training_accepted_filename, index=False)

    # select validation predictions meeting the retention threshold
    validation_retained_predictions = validation_predictions[
        validation_predictions["Probability CUI In The Image"] >= pipeline_retention_threshold
```

```
    ].copy()

    # reapply symbolic and conformal validation to retained validation predictions
    validation_retained_pipeline = pipeline_controller.create_pipeline(
        validation_retained_predictions,
        symbolic_validator,
        conformal_calibrator
    )

    # retain reliable validation predictions
    validation_accepted_pipeline = validation_retained_pipeline[
        validation_retained_pipeline["Reliable"]
    ].reset_index(drop=True)

    # save accepted validation predictions separately
    validation_accepted_filename = Path(pipeline_output_folder) / "pipeline_validation_accepted.csv"
    validation_accepted_pipeline.to_csv(validation_accepted_filename, index=False)

    # display saved files
    print(training_pipeline_filename)
    print(validation_pipeline_filename)
    print(training_accepted_filename)
    print(validation_accepted_filename)
```

In [63]:

```
# export test pipeline CSV files
if pipeline_mode == "test":

    # export complete test pipeline including rejected predictions
    test_pipeline_filename = pipeline_controller.save_pipeline(
        test_pipeline,
        pipeline_output_folder,
        "test"
    )

    # select test predictions meeting the retention threshold
    test_retained_predictions = test_predictions[
        test_predictions["Probability CUI In The Image"] >= pipeline_retention_threshold
    ].copy()

    # reapply symbolic and conformal validation to retained test predictions
    test_retained_pipeline = pipeline_controller.create_pipeline(
```

```
        test_retained_predictions,
        symbolic_validator,
        conformal_calibrator
    )

    # retain reliable test predictions
    test_accepted_pipeline = test_retained_pipeline[
        test_retained_pipeline["Reliable"]
    ].reset_index(drop=True)

    # save accepted test predictions separately
    test_accepted_filename = Path(pipeline_output_folder) / "pipeline_test_accepted.csv"
    test_accepted_pipeline.to_csv(test_accepted_filename, index=False)

    # display saved files
    print(test_pipeline_filename)
    print(test_accepted_filename)
```

```
ROCOv2\pipeline_test.csv
ROCOv2\pipeline_test_accepted.csv
```

Validate Pipeline Outputs

In [64]:

```
def validate_pipeline_output(pipeline, expected_image_ids, split_name):
    """
    Validate final pipeline structure and values

    Args:
        pipeline (DataFrame): final pipeline data
        expected_image_ids: image IDs belonging to the split
        split_name (str): display name

    Returns:
        dict: validation summary
    """

    try:

        # required output columns
        required_columns = ["Image ID", "CUI", "Concept Name", "Probability CUI In The Image", "Symbolic (Valid)",
"Conformal (Valid)", "Reliable"]
```

```
        # check required columns
        if list(pipeline.columns) != required_columns:
            raise ValueError(f"{split_name} pipeline columns do not match the required output structure")

        # check probabilities
        if not pipeline["Probability CUI In The Image"].between(0, 1).all():
            raise ValueError(f"{split_name} pipeline contains probabilities outside 0 to 1")

        # check duplicate image/CUI rows
        if pipeline.duplicated(subset=["Image ID", "CUI"]).any():
            raise ValueError(f"{split_name} pipeline contains duplicate image/CUI rows")

        # check image IDs belong to expected split
        unexpected_image_ids = set(pipeline["Image ID"].astype(str)) - set(str(image_id) for image_id in
expected_image_ids)
        if unexpected_image_ids:
            raise ValueError(f"{split_name} pipeline contains unexpected image IDs")

        # check reliability logic
        expected_reliable = pipeline["Symbolic (Valid)"] & pipeline["Conformal (Valid)"]
        if not pipeline["Reliable"].equals(expected_reliable):
            raise ValueError(f"{split_name} pipeline contains invalid Reliable values")

        # create validation summary
        validation_summary = {"Split": split_name, "Expected Images": len(set(expected_image_ids)), "Images With
Candidate Concepts": pipeline["Image ID"].nunique(), "Rows": len(pipeline), "Unique CUIs":
pipeline["CUI"].nunique()}

        return validation_summary

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [65]:

```
# validate pipeline outputs
validation_results = []
```

In [66]:
```python
if pipeline_mode == "train_val":

    # validate training and validation pipeline outputs
    validation_results.append(validate_pipeline_output(training_pipeline, training_ids, "Training"))
    validation_results.append(validate_pipeline_output(validation_pipeline, validation_ids, "Validation"))
```

In [67]:
```python
if pipeline_mode == "test":

    # validate test pipeline output
    validation_results.append(validate_pipeline_output(test_pipeline, test_ids, "Test"))
```

In [68]:
```python
# display validation results
display(pd.DataFrame(validation_results))
```

	Split	Expected Images	Images With Candidate Concepts	Rows	Unique CUIs
0	Test	9927	9927	1823618	1766

Pipeline Statistics

In [69]:
```python
def calculate_pipeline_statistics(pipeline, split_name):
    """
    Calculate pipeline acceptance statistics

    Args:
        pipeline (DataFrame): final pipeline data
        split_name (str): display name

    Returns:
        dict: pipeline statistics
    """

    try:

        # calculate counts
        prediction_rows = len(pipeline)
        symbolic_valid = int(pipeline["Symbolic (Valid)"].sum()) if prediction_rows > 0 else 0
        conformal_valid = int(pipeline["Conformal (Valid)"].sum()) if prediction_rows > 0 else 0
        reliable = int(pipeline["Reliable"].sum()) if prediction_rows > 0 else 0
```

```
        # calculate percentages
        symbolic_acceptance = (symbolic_valid / prediction_rows) * 100 if prediction_rows > 0 else 0.0
        conformal_acceptance = (conformal_valid / prediction_rows) * 100 if prediction_rows > 0 else 0.0
        reliable_percentage = (reliable / prediction_rows) * 100 if prediction_rows > 0 else 0.0

        # return statistics
        return {"Split": split_name, "Prediction Rows": prediction_rows, "Images": pipeline["Image ID"].nunique(),
"Unique CUIs": pipeline["CUI"].nunique(), "Mean Probability": pipeline["Probability CUI In The Image"].mean() if
prediction_rows > 0 else 0.0, "Symbolic Valid": symbolic_valid, "Symbolic Acceptance %": symbolic_acceptance,
"Conformal Valid": conformal_valid, "Conformal Acceptance %": conformal_acceptance, "Reliable": reliable, "Reliable
%": reliable_percentage}

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [70]:
```
# calculate pipeline statistics
pipeline_statistics = []
```

In [71]:
```
if pipeline_mode == "train_val":

    # calculate training and validation statistics
    pipeline_statistics.append(calculate_pipeline_statistics(training_pipeline, "Training"))
    pipeline_statistics.append(calculate_pipeline_statistics(validation_pipeline, "Validation"))
```

In [72]:
```
if pipeline_mode == "test":

    # calculate test statistics
    pipeline_statistics.append(calculate_pipeline_statistics(test_pipeline, "Test"))
```

In [73]:
```
# display pipeline statistics
display(pd.DataFrame(pipeline_statistics))
```

	Split	Prediction Rows	Images	Unique CUIs	Mean Probability	Symbolic Valid	Symbolic Acceptance %	Conformal Valid	Conformal Acceptance %	Reliable	Reliable %
0	Test	1823618	9927	1766	0.679922	1823395	99.987772	681417	37.366214	681398	37.365172

References

BiomedCLIP-PubMedBERT_256-vit_base_patch16_224: https://huggingface.co/microsoft/BiomedCLIP-PubMedBERT_256-vit_base_patch16_224

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

ROCOv2: Radiology Objects in Context Version 2, An Updated Multimodal Image Dataset: https://doi.org/10.1038/s41597-024-03496-6